Time Complexity

Lesson & Problems

Motivation 🤔

Let's look at the subtasks of some IOI problem (IOI 2024 - Mosaic):

Subtask	Score	Additional Constraints
1	5	$N \leq 2; Q \leq 10$
2	7	$N \leq 200; Q \leq 200$
3	7	$T[k] = B[k] = 0$ (for each k such that $0 \leq k < Q$)
4	10	$N \leq 5000$
5	8	$X[i] = Y[i] = 0$ (for each i such that $0 \leq i < N$)
6	22	$T[k] = B[k]$ and $L[k] = R[k]$ (for each k such that $0 \leq k < Q$)
7	19	$T[k] = B[k]$ (for each k such that $0 \leq k < Q$)
8	22	No additional constraints.

We can see that solving the problem for $N \leq 5000$ gives a certain amount of points, but not a full mark as in general $N \leq 200\,000$. So how would some algorithm solve the problem for $N \leq 5000$ but not $N \leq 200\,000$? 🤔

The main thing to think about is the restriction in terms of time ⏳ and memory 💾. It is always mentioned either in the problem statement or the online judge that there is a limit of time and memory (For this example, the time limit is 2.0 seconds and the memory limit is 1 gigabyte). So our algorithm has to be fast enough and use a minimal amount of memory in order to pass all the test cases without exceeding the time and memory limits. The time limit is generally the main issue. Can you estimate how much time your algorithm is going to run without trying it?

Actually, yes! Let's see how to do it! 🥳

Big-O Notation 💡

We can estimate the speed of an algorithm through its time complexity. Suppose that we have an algorithm that relies on some integer parameters $p_1, p_2, \ldots, p_k$. Then its time complexity is denoted by $O(f(p_1, p_2, \ldots, p_k))$ for some function $f : (p_1, p_2, \ldots, p_k) \to N$. Saying that our algorithm runs in $O(f(p_1, p_2, \ldots, p_k))$ means that the number of operations it takes is at most $C \times f(p_1, p_2, \ldots, p_k)$ for some constant C.

It may seem a little complicated, so let's start with a simple example:

```cpp
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n;
    cin >> n;
    for(int i = 0; i < n; i++){
        cout << i << '\n';
    }
}
```

We can see that our algorithm takes one parameter, which is n, and then runs a loop n times to show the numbers from 0 to $n-1$. Hence, it executes n operations. Therefore, our algorithm runs in $O(n)$.

Let's try with a second one:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n;
    cin >> n;
    for(int i = 0; i < 3*n; i++){
        cout << i << '\n';
    }
}
```

This algorithm shows us the numbers from 0 to $3n-1$, hence we have $3n$ operations. Someone might think that the algorithm runs in $O(3n)$. However, according to the definition we stated before, an algorithm running in $O(f(n))$ only means that the algorithm executes $C \times f(n)$ operations for some constant C. Hence, by putting $C = 3$, our algorithm runs in $O(n)$. By convention, we don't leave any constant factor in the big-O notation. Hence, instead of $O(3n)$, we always write $O(n)$, even though you SHOULD keep in mind the constant factor, especially when it is large. Did you guess this one?

Let's move to a third example:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n, m;
    cin >> n >> m;
    for(int i = 0; i < 3*n; i++){
        for(int j = 0; j < 10*m; j++){
            cout << (i*j) % 57 << '\n';
        }
    }
}
```

Try to compute the time complexity of this one yourself, then read the explanation below. Good luck! 😉

Notice that we have a loop executing $10m$ times the code `cout << (i*j)%57 << '\n';`, which consists of 3 operations (multiplication, then modulo, then output to console). This loop is executed $3n$ times. Thus, we have $3n \times 10m \times 3$ operations, giving us a complexity of $O(nm)$ as we are ignoring constant terms. Was your answer correct? 🙃

Now, what happens when I execute multiple blocks of code one after the other with different complexities? 🤔

Well, the rule is simple: $O(f(n) + g(n)) = O(\max(f(n), g(n)))$ for n large enough. In other words, we take the complexity of the slowest code block!

Example: Let's just put the examples 1 and 3 together:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n, m;
    cin >> n >> m;
    for(int i = 0; i < n; i++){
        cout << i << '\n';
    }

    for(int i = 0; i < 3*n; i++){
        for(int j = 0; j < 10*m; j++){
            cout << (i*j) % 57 << '\n';
        }
    }
}
```

In the first block, we have a complexity of $O(n)$, and in the second, a complexity of $O(nm)$. It is obvious that the second block will take more time, as when taking n and m large enough, $nm > n$. 💯

After seeing complexity classes, you will find that comparing complexities is very easy. Don't worry! But let's study another case before going to complexity classes:

You probably know about the Fibonacci Sequence, which was discovered by *Leonardo Fibonacci* (Fun fact: he actually studied in Algeria 🇩🇿). The sequence has the following formula:

$$F_n = F_{n-1} + F_{n-2}$$

with $F_0 = 1$ and $F_1 = 1$. Let's make a function to compute it:

```
#include <bits/stdc++.h>
using namespace std;
int F(int n){
    if(n < 2) return 1;
    return F(n-1) + F(n-2);
}
```

Let $f(n)$ be the number of operations needed to compute F_n. Then $f(0) = f(1) = 1$, since it only needs one operation. For n larger, $f(n) = f(n-1) + f(n-2)$. For the sake of simplicity, for n large enough, consider that $f(n-2) = f(n-1)$. Hence, $f(n) = 2 \times f(n-1)$, giving $f(n) = 2^n$. Therefore, we can consider our algorithm running in $O(2^n)$, which is an exponential complexity.

Sorry for seeming like I'm doing forbidden math through approximations 😅, but in competition, you won't need to compute an exact complexity. Instead, you only need to estimate it, as you don't have a lot of time, and a quick, good-enough estimation will help you more than an exact one, which will take longer to compute.

Complexity Classes (Or How to Know if My Algorithm is Fast Enough) ⏱️

If you bought your computer, you probably saw in its specifications the clock speed of the CPU. But did you ever wonder what it means? 🧐

You can see on the CPU "3.70 GHz". This actually means that the CPU can run 3.70×10^9 operations per second. This is great information to estimate the time an algorithm will take to run! The thing is that since C++ code isn't interpreted directly by the CPU, there's a constant factor to consider. So here's a table that will help you estimate the time complexity you will need depending on the size of your input:

Disclaimer: This table is NOT to be memorized, as you will naturally learn which complexity you need by practicing on more and more problems. I am providing this table only to help you do estimations at the beginning of your journey in CP.

Time Complexity	**Size of the Input**
$O(\log n)$	n is large
$O(\sqrt{n})$	$n \leq 10^{14}$
$O(n)$	$n \leq 10^7$
$O(n \log n)$	$n \leq 10^6$
$O(n^2)$	$n \leq 5000$
$O(n^3)$	$n \leq 500$
$O(2^n)$	$n \leq 20$
$O(n!)$	$n \leq 10$

Do not forget that the time complexity is an estimation due to the constant factor we ignore when computing! And that this table isn't absolute!

Advice: If your estimation of the number of operations is very close to the estimated limit of 5×10^7 operations per second, there's still hope for your solution to work, as complexity is often an estimation. Give your solution a try and hope that it doesn't get a TLE (Time Limit Exceeded 😢)!

On the $O(\log n)$ Complexity 🍰

Most likely, you will encounter this complexity when dealing with algorithms that divide the problem into two parts each time, running approximately $C \times \log_2 n$ operations.

Example: Suppose that you have a piece of cake 🍰 of length n centimeters, and you put it on a table. Then you keep doing the following operation: Take the leftmost piece of cake on the table and cut it equally into two parts, until the leftmost piece is less than 1 cm long. Hence, you will have to do around $\log_2 n$ operations.

For $n = 4$**:** $4 \to 2 \to 1 \to 0.5$
For $n = 12$**:** $12 \to 6 \to 3 \to 1.5 \to 0.75$
For $n = 16$**:** $16 \to 8 \to 4 \to 2 \to 1 \to 0.5$

You will generally find this complexity when dealing with the paradigm of Divide-and-Conquer, sortings, and binary search. I won't spoil the fun here, haha!

Problems

Estimate the time complexity of the following functions:

```
#include <iostream>
using namespace std;

bool isPrime(int n) {
    if (n <= 1) return false;
    for (int i = 2; i * i <= n; i++) {
        if (n % i == 0) return false;
    }
    return true;
}
```

```
#include <iostream>
using namespace std;
int findMax(int arr[], int n) {
    int maxVal = arr[0];
    for (int i = 1; i < n; i++) {
        if (arr[i] > maxVal) {
            maxVal = arr[i];
        }
    }
    return maxVal;
}
```

```
#include <iostream>
using namespace std;

int power(int x, int n) {
    int result = 1;
    while (n > 0) {
        if (n % 2 == 1)
            result *= x;
        x *= x;
        n /= 2;
    }
    return result;
}
```

```
#include <iostream>
using namespace std;

void prefixSum(int arr[], int n) {
    int prefix[n];
    prefix[0] = arr[0];

    for (int i = 1; i < n; i++) {
        prefix[i] = prefix[i - 1] + arr[i];
    }
```

```
    for (int i = 0; i < n; i++) {
        cout << prefix[i] << " ";
    }
    cout << endl;
}
```

```
#include <iostream>
using namespace std;

void findDivisors(int n) {
    for (int i = 1; i * i <= n; i++) {
        if (n % i == 0) {
            cout << i << " ";
            if (i != n / i) {
                cout << n / i << " ";
            }
        }
    }
    cout << endl;
}
```

```
#include <iostream>
using namespace std;

void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

```
#include <iostream>
using namespace std;

int find(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target)
            return mid;
        else if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
```

```cpp
#include <iostream>
#include <vector>
using namespace std;

void generateSubsets(vector<int>& subset, int arr[], int n, int index) {
    if (index == n) {
        for (int val : subset)
            cout << val << " ";
        cout << endl;
        return;
    }

    // Exclude the current element
    generateSubsets(subset, arr, n, index + 1);

    // Include the current element
    subset.push_back(arr[index]);
    generateSubsets(subset, arr, n, index + 1);

    subset.pop_back(); // Backtrack
}
```

```cpp
#include <iostream>
#include <cmath>
using namespace std;

void generateNumbers(int n) {
    int start = pow(10, n - 1); // Smallest n-digit number
    int end = pow(10, n) - 1; // Largest n-digit number

    for (int i = start; i <= end; i++) {
        cout << i << " ";
    }
    cout << endl;
}
```

Answers ✓

Do not read the answers before trying. Share your mark on the AOI Discord with your coach.

1. $O(\sqrt{n})$
2. $O(n)$
3. $O(\log n)$
4. $O(n)$
5. $O(\sqrt{n})$
6. $O(n^2)$
7. $O(\log n)$
8. $O(2^n)$
9. $O(10^n)$